UACM

Universidad Autónoma de la Ciudad de México

NADA HUMANO ME ES AJENO

JUEGO_DE_CARTAS (Construcción y evolución de un juego de cartas en Java)

Inteligencia de negocios

Licenciatura en Ingeniería de Software

ANGEL ANTONIO GARCIA QUINTERO
LESLIE JOSELIN AGUILAR DOMINGUEZ
CARMEN ANDREA PEREZ COPADO
IVAN ENRIQUE RAMIREZ GALLARDO

HISTORIAL DE VERSIONES

Fecha	Versión	Descripción	Autores
24/02/26	1.0	Se entrega el avance de documentación y avance de código	ANGEL ANTONIO GARCIA QUINTERO LESLIE JOSELIN AGUILAR DOMINGUEZ CARMEN ANDREA PEREZ COPADO IVAN ENRIQUE RAMIREZ GALLARDO

Índice

INTRODUCCION 3

Misión 3

Visión 3

Alcance 3

Roles 4

Requisitos funcionales 4

Requisitos no funcionales 5

Diagrama de caso de uso/secuencia 6

Diagrama UML 7

Clases 8

Relaciones UML 9

CODIGO 10

- Main 10
- Game 10
- Hand 12
- Card 13
- Deck 14

Entrega 15

INTRODUCCION

Desarrollar una aplicación en Java que implemente una versión básica del juego UNO en consola, permitiendo a un jugador humano competir contra la computadora utilizando únicamente cartas numéricas.
El sistema servirá como base evolutiva para versiones posteriores del proyecto.

Misión

Desarrollar un juego de cartas llamado JUEGO_DE_CARTAS en Java, con interfaz de consola, que permita una partida entre un jugador humano y la computadora siguiendo las reglas básicas del Uno (coincidencia de color o número). El sistema creará una baraja completa con 4 colores y números del 0 al 9, la barajeara, repartirá 7 cartas a cada participante y colocará una carta inicial en la mesa. Durante el juego, el jugador podrá seleccionar cartas por índice, y la computadora jugará automáticamente su primera carta válida. Ambos deberán robar cuando no tengan jugadas posibles. El proyecto se construirá aplicando programación orientada a objetos, con clases separadas, código comentado y legible, y gestionado mediante control de versiones con Git, cumpliendo así los requisitos funcionales y no funcionales establecidos en la materia de Construcción y Evolución de Software. El juego finalizará mostrando un mensaje de victoria para el jugador o la computadora cuando uno se quede sin cartas o ya no haya más cartas que robar.

Visión

Convertirse en un proyecto de referencia dentro de la asignatura que demuestre buenas prácticas de ingeniería de software: modularidad, mantenibilidad y facilidad de evolución. A futuro, se espera que JUEGO_DE_CARTAS pueda extenderse con nuevas funcionalidades como cartas especiales (reversa, bloqueo, toma dos, comodines), soporte para más jugadores e incluso una interfaz gráfica, sin necesidad de reestructurar su núcleo. De esta forma, el proyecto servirá como base didáctica para enseñar conceptos de diseño, evolución y mantenimiento de software, mostrando cómo una arquitectura sólida permite adaptarse a nuevos requerimientos de manera eficiente y ordenada.

Alcance

En esta primera versión incluirá únicamente juego de la consola, las cartas numéricas (0-9) , la distinción de los cuatro colores diferentes de cada carta, el funcionamiento de las reglas básicas del juego del uno , se tendrá solo contemplado un jugador y jugador computadora mostrando un flujo completo del juego.

Roles

Programador principal	**Ángel Antonio García Quintero**
QA/testing	Ramírez Gallardo Iván Enrique
Documentador	Aguilar Domínguez Leslie Joseline
Integrador	Pérez Copado Carmen Andrea
Desarrollador	Pérez Copado Carmen Andrea Aguilar Domínguez Leslie Joseline Ramírez Gallardo Iván Enrique Ángel Antonio García Quintero

Requisitos funcionales

Id	Modulo	Requisito funcional
RF1	Creación de la baraja	El sistema deberá crear una baraja con: • 4 colores:rojo azul,verde,amarillo • Números de 0 a 9 • Dos cartas por numero por color • El sistema barajeara la baraja al iniciar
RF2	Reparto de cartas	El sistema debera: • Repartir 7 cartas al jugador • Repartir 7 cartas a la computadora • Colocar una carta inicial en mesa
RF3	Visualización en consola	El Sistema deberá mostrar: • Carta actual en mesa • Cartas del jugador con índice • Mensaje de tirno • Acciones realizadas
RF4	Turno del jugador	El Sistema deberá permitir al jugador: • Seleccionar carta por índice • Coincide con el numero Si no tiene jugada valida: • Deberá robar automáticamente

RNF5	Turno jugador computadora	El sistema debera: • Buscar primera carta valida • Jugarla automáticamente • Sino tiene carta valida ➞robar
RNF6	Validación de jugadas	El sistema debera verificar que cada una de las cartas pueda jugarse si: • Coincide el color con carta en mesa o • Coincide el numero Si no es válida: • Se regresa a la mano • Se solicita otra acción
RNG7	Condición de victoria	El sistema deberá finalizar cuando: • El jugador se queda sin cartas • Ya no hay cartas para recoger Mostrar mensaje: • "Jugador gana" • "Computadora gana"

Requisitos no funcionales

Id	Modulo	Requisito funcional
RNF1	Lenguaje	El sistema deberá desarrollar en java
RNF2	Interface	La interface solo será por consola
RNF3	Diseño	El sistema deberá : Usar programación orientada a objetos Tener clases separadas Ser legible y maleable
RNF4	Calidad de código	El código deberá: Estar comentado Tener nombres claros Evitar código monolítico
RNF5	Control de versiones	El proyecto deberá almacenarse en Git.

Diagrama de caso de uso/secuencia

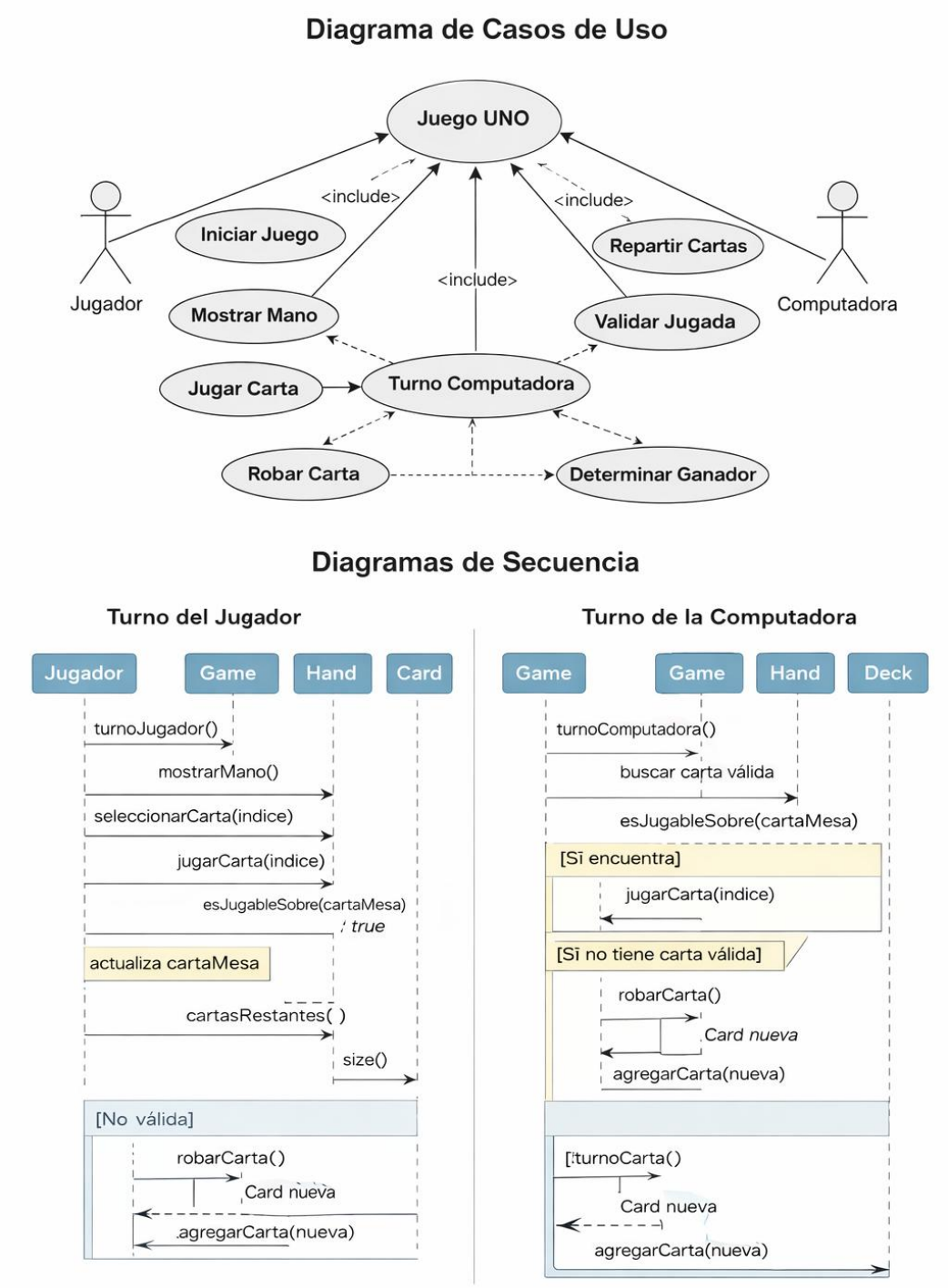
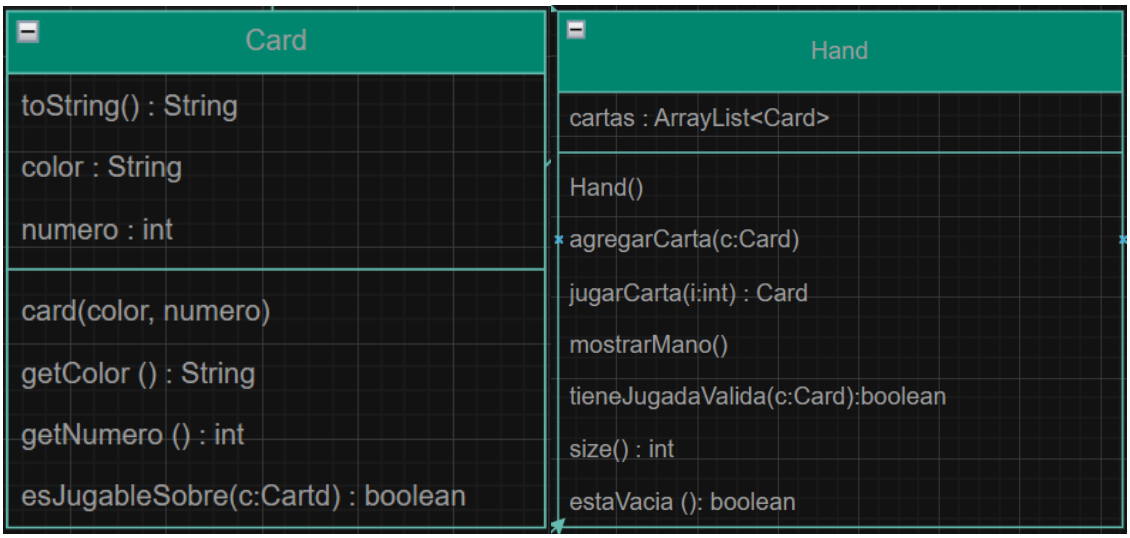
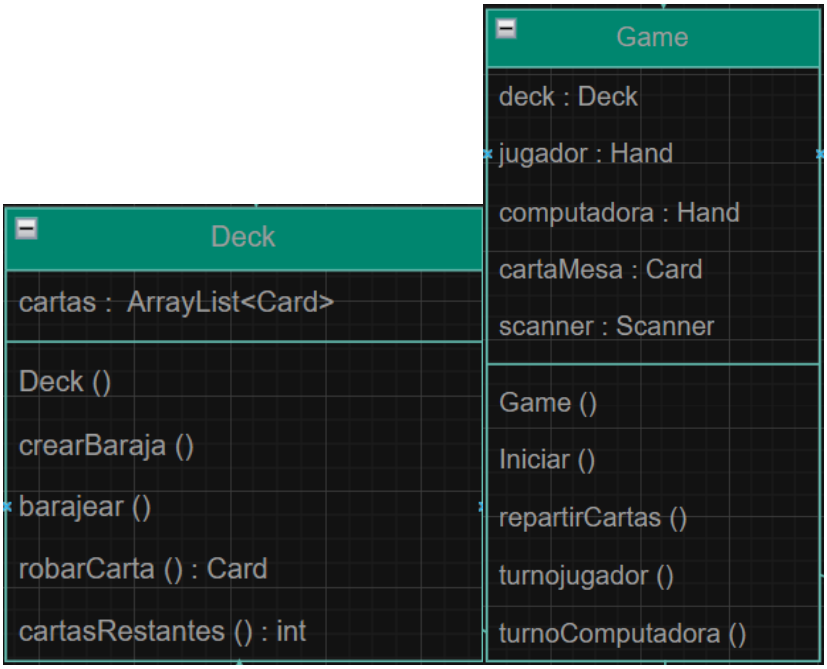


Diagrama UML

Clase atributos/métodos			
	Relación	Tipo UML	Ejemplo
Usa	Game➞Deck	Asociación	Una baraja
Usa	Game➞Hand	Asociación	Controla mano
Usa	Game➞Card	Asociación	
Crear	Deck➞Card	Composición	Baraja tiene cartas
Contiene	Hand➞Card	Composición	Mano tiene cartas
Crear	Main➞Game	Dependencia	Lo ejecuta

Clases


Card
toString() : String
color : String
numero : int
card(color, numero)
getColor () : String
getNumero () : int
esJugableSobre(c:Cartd) : boolean
Hand
cartas : ArrayList<Card>
Hand()
agregarCarta(c:Card)
jugarCarta(i:int) : Card
mostrarMano()
tieneJugadaValida(c:Card):boolean
size() : int
estaVacia (): boolean


Game
deck : Deck
jugador : Hand
computadora : Hand
cartaMesa : Card
scanner : Scanner
Game ()
Iniciar ()
repartirCartas ()
turnojugador ()
turnoComputadora ()
Deck
cartas : ArrayList<Card>
Deck ()
crearBaraja ()
barajear ()
robarCarta () : Card
cartasRestantes () : int

Main
main

Relaciones UML

CODIGO

Main

```
package JUEGO_DE_CARTAS;

public class Main {

    public static void main(String[] args) {

        Game juego = new Game();
        juego.iniciar();

    }
}
```

Game

```
package JUEGO_DE_CARTAS;

import java.util.Scanner;

public class Game {

      private Deck deck;
      private Hand jugador;
      private Hand computadora;
      private Card cartaMesa;
      private Scanner scanner;

      public Game() {
            deck = new Deck();
            jugador = new Hand();
            computadora = new Hand();
            scanner = new Scanner(System.in);
      }

      public void iniciar() {

            repartirCartas();

            // Colocar primera carta en la mesa
            cartaMesa = deck.robarCarta();
            System.out.println("Carta inicial en la mesa: " + cartaMesa);

            // Ciclo del juego
            while (true) {

                  turnoJugador();
                  if (jugador.estaVacia()) {
                        System.out.println("\n(⌐■_■) ¡Ganaste
FELICIDADES!");
```

```
                        break;
                    }

                    turnoComputadora();
                    if (computadora.estaVacia()) {
                        System.out.println("\n💻 La computadora ganó .");
                        break;
                    }
                }
            }

            private void repartirCartas() {
                for (int i = 0; i < 7; i++) {
                    jugador.agregarCarta(deck.robarCarta());
                    computadora.agregarCarta(deck.robarCarta());
                }
            }

            private void turnoJugador() {
                System.out.println("\n--- Tu turno ---");
                System.out.println("Carta en mesa: " + cartaMesa);

                jugador.mostrarMano();

                if (!jugador.tieneJugadaValida(cartaMesa)) {
                    System.out.println("No tienes jugadas válidas. Robas
una carta.");
                    jugador.agregarCarta(deck.robarCarta());
                    return;
                }

                System.out.print("Elige índice de carta para jugar o -1 para
robar: ");
                int opcion = scanner.nextInt();

                if (opcion == -1) {
                    Card nueva = deck.robarCarta();

                    if (nueva != null) {
                        jugador.agregarCarta(nueva);
                        System.out.println("Robaste: " + nueva);
                    } else {
                        System.out.println("No hay más cartas en el mazo.");
                    }
                    return;
                }

                Card cartaElegida = jugador.jugarCarta(opcion);

                if (cartaElegida == null) {
                    System.out.println("Índice inválido.");
                    return;
                }

                if (cartaElegida.esJugableSobre(cartaMesa)) {
                    cartaMesa = cartaElegida;
                    System.out.println("Jugaste: " + cartaMesa);
```

```
            } else {
                System.out.println("Movimiento inválido.");
                jugador.agregarCarta(cartaElegida); // solo si NO es null
            }
        }

        private void turnoComputadora() {
            System.out.println("\n--- Turno computadora ---");

            for (int i = 0; i < computadora.size(); i++) {

                Card carta = computadora.jugarCarta(i);

                if (carta != null && carta.esJugableSobre(cartaMesa)) {
                    cartaMesa = carta;
                    System.out.println("La computadora jugó: " +
cartaMesa);
                    return;
                } else {
                    // si no sirve, la regresamos
                    computadora.agregarCarta(carta);
                }
            }

            // Si no puede jugar, roba
            Card nueva = deck.robarCarta();

            if (nueva != null) {
                computadora.agregarCarta(nueva);
                System.out.println("La computadora roba una carta.");
            } else {
                System.out.println("No hay más cartas en el mazo.");
            }
        }
}
```

Hand

```
package JUEGO_DE_CARTAS;

import java.util.ArrayList;


/**
 * Representa la mano de un jugador
 */
public class Hand {

    private ArrayList<Card> cartas;

    public Hand() {
        cartas = new ArrayList<>();
    }

    public void agregarCarta(Card c) {
        cartas.add(c);
    }
```

```
    public Card jugarCarta(int indice) {
        if (indice < 0 || indice >= cartas.size()) return null;
        return cartas.remove(indice);
    }

    public int size() {
        return cartas.size();
    }

    public boolean estaVacia() {
        return cartas.isEmpty();
    }

    public void mostrarMano() {
        System.out.println("Cartas en mano:");
        for (int i = 0; i < cartas.size(); i++) {
            System.out.println(i + ": " + cartas.get(i));
        }
    }

    /**
     * Verifica si tiene al menos una jugada válida
     */
    public boolean tieneJugadaValida(Card cartaMesa) {
        for (Card c : cartas) {
            if (c != null && c.esJugableSobre(cartaMesa)) {
                return true;
            }
        }
        return false;
    }

}
```

Card

```
package JUEGO_DE_CARTAS;

public class Card {

      private String color;
      private int numero;

      public Card(String color, int numero) {
            this.color = color;
            this.numero = numero;
      }

      public boolean esJugableSobre(Card otra) {
            if (otra == null) return true;
            return this.color.equals(otra.color) || this.numero ==
otra.numero;
      }

      @Override
      public String toString() {
```

```java
            return color + " " + numero;
        }

        public String getColor() {
            return color;
        }

        public int getNumero() {
            return numero;
        }
}
```

Deck

```java
package JUEGO_DE_CARTAS;

import java.util.ArrayList;
import java.util.Collections;
/**
 * Representa la baraja basica de UNO (sin cartas especiales)
 * */
public class Deck{

    private ArrayList<Card>cartas;
    public Deck() {
        cartas = new ArrayList<>();
        crearBaraja();
        barajar();
    }
/**
 * Crea baraja básica:
 * 4 colores, números 0-9, dos de cada número
 */
    private void crearBaraja() {
        String[] colores = {"rojo", "azul", "verde", "amarillo"};

        for (String color : colores) {
            for (int i = 0; i <= 9; i++) {
                cartas.add(new Card(color, i));
                cartas.add(new Card(color, i)); // dos de cada
            }
        }
    }

    public void barajar() {
    Collections.shuffle(cartas);
    }

    public Card robarCarta() {
    if (cartas.isEmpty()) return null;
    return cartas.remove(0);
    }

    public int cartasRestantes() {
    return cartas.size();
    }
}
```

Entrega

SEMANAS 1–2

Versión 1:

• Base funcional • Desarrollar en consola.

Clases mínimas:

• Carta • Baraja • Jugador • Juego • Main

Requerimientos:

• Reglas básicas del juego UNO• Repartir cartas • Turnos • Validar jugadas • Determinar ganador

Entregar:

• UML inicial • v1 funcional • repositorio Git